

- Hilary AI: Deep Technical Systems & Architecture Analysis
 - 1. System Activity Flow Model: Deep Process Breakdown
 - Phase 1: Authentication & Access (Auth -> Dashboard)
 - Phase 2: The Multi-Modal Therapy Loop (Chat State)
 - Phase 3: Analytics Retrieval (Insights State)
 - 2. Entity Relationship Diagram (ERD): Database Normalization
 - USER Entity (The Root)
 - CHAT_MESSAGE Entity (The Temporal Ledger)
 - BEHAVIORAL_DATA Entity (Passive Telemetry)
 - 3. System Class Diagram: Architectural Components
 - Frontend Classes
 - Backend Services

Hilary AI: Deep Technical Systems & Architecture Analysis

This document provides a granular, node-by-node, class-by-class technical breakdown of the architecture underlying the Hilary AI Therapy Platform. This references the three primary architectural diagrams: The System Activity Flow Model, The Entity-Relationship Diagram (ERD), and the System Class Diagram.

1. System Activity Flow Model: Deep Process Breakdown

The Activity Flow Model illustrates the chronological and conditional execution paths during user interaction.

Phase 1: Authentication & Access (Auth -> Dashboard)

- **Node [*] --> Auth:** The entry point. The client sends a **POST /auth/login** containing **username** (email) and **password** via **application/x-www-form-**

`urlencoded`. The FastAPI backend verifies the bcrypt hashed password against the SQLite/PostgreSQL `User` table.

- **Transition `Auth` --> `Dashboard`:** Upon successful validation, the server issues a JWT (`access_token`) with a 1-week expiry. The client stores this in `localStorage` and transitions the React router state to the `Dashboard`.

Phase 2: The Multi-Modal Therapy Loop (`Chat State`)

This state governs the core interaction mechanism, handling the complex fusion of different data streams.

- **Node `MultiModalInput`:** The React UI (`ChatApp.jsx`) intercepts the user's `onSubmit` event. It captures the text string. If an image file is attached via the `<input type="file">`, the `FileReader` API asynchronously converts the blob to a Base64 string.
- **Node `DataPreprocessing`:** The client constructs a JSON payload containing the last 10 messages (for context), a default `text_sentiment` (0.0), and the `image_b64` string. This is dispatched via `apiClient.post('/chat/')`.
- **State `Custom_Hilary_Model`:** This is where the backend AI processing occurs.
 - **Node `FeatureExtraction`:** In `ai_service.py`, if `image_b64` is present, it is decoded back into binary, converted to a PIL `Image`, and passed through `torchvision.transforms` (resize to 256, center crop to 224, normalized via standard ImageNet metrics). The resulting tensor is pushed to the GPU/CPU and fed into `mental_health_image_model.pt`.
 - **Node `CognitiveLayer`:** The PyTorch model outputs a logit tensor. We apply an `argmax` function to map this to an emotion (e.g., "Sad", "Anxious"). Concurrently, the text undergoes a heuristic keyword analysis to establish a baseline `text_sentiment` score (-1.0 to 1.0).
 - **Node `EmotionClassification`:** The `EmotionEngine` calculates a weighted sum of the behavioral data (screen time), the PyTorch visual sentiment, and the text sentiment to produce a `preliminary_state`. This state is injected into the System Prompt. The Groq API (`llama-3.3-70b-versatile`) is invoked in strict JSON mode to produce the final therapeutic response and refine the `detected_sentiment`.
- **Node `ReturnResponse`:** The Groq API returns a JSON object containing the text response, intensity score, and clinical insights.

- **Node `UpdateHistory`**: The backend instantiates new `ChatMessage` ORM objects for both the user input (including the `image_preview` base64 if desired) and the assistant's response, committing them to the SQL database via `session.commit()`.
- **Node `CheckDistress & TriggerAlert`**: The `AlertService` inspects the final `detected_sentiment`. If the state matches the string "`Critical Distress`", it triggers the UI to render the `showCrisisModal` overlay, interrupting the session with emergency hotline resources (e.g., 988 Lifeline).

Phase 3: Analytics Retrieval (**Insights** State)

- **Node `FetchData`**: The client triggers a `GET /dashboard/summary` request.
 - **Node `CalculateMetrics & RenderGraphs`**: The backend aggregates the raw `BEHAVIORAL_DATA` over time intervals, returning structured statistics. The React frontend maps this data onto CSS-based dynamic bar charts, calculating width percentages dynamically based on the frequency of states like "Focus" vs "Distraction."
-

2. Entity Relationship Diagram (ERD): Database Normalization

The ERD maps the strict `SQLModel` (`SQLAlchemy`) relational structure that guarantees data integrity and historical context.

USER Entity (The Root)

- `id` (Integer/PK): The primary foreign key target for all associated records.
- `email` (String/UK): Unique constraint enforced at the DB level to prevent duplicate registrations.
- `hashed_password` (String): Securely stores the bcrypt output; plaintext passwords never enter the DB.
- `is_verified` (Boolean): A flag designed for email verification workflows.

CHAT_MESSAGE Entity (The Temporal Ledger)

This table acts as a highly detailed audit log of every interaction.

- `id` (Integer/PK)
- `user_id` (Integer/FK): Maps `N:1` back to the `USER` table.
- `role` (String): Strict enforcement of either "user" or "assistant".
- `content` (Text): The actual message payload.
- `image_ref` (String): Stores the base64 string or a file path to the uploaded visual modality.
- `emotional_state` (String): Captures the precise output of the `EmotionEngine` *at the time the message was sent*, creating a historical timeline of mood fluctuations.
- `insights` (Text): Stores the private clinical observations generated by the Groq model regarding the patient's cognitive state.
- `timestamp` (DateTime): Defaults to `datetime.utcnow()`.

BEHAVIORAL_DATA Entity (Passive Telemetry)

Stores passive device metrics used to influence the emotional baseline.

- `id` (Integer/PK)
- `user_id` (Integer/FK): Maps `N:1` back to `USER`.
- `screen_time_seconds` (Integer): Represents total engagement time. The `EmotionEngine` flags values over 21,600 (6 hours) as indicators of potential anxiety/stress.
- `unlock_count` (Integer): Represents device checking frequency. Values over 100 trigger stress heuristics.

3. System Class Diagram: Architectural Components

The Class diagram reveals the specific abstractions and dependency injection used across the codebase.

Frontend Classes

- **App**: The root React component. Manages the global authentication state (`isAuthenticated`, `user`). It conditionally renders either the `Auth` module or the `ChatApp`.
- **ChatApp**: A monolithic state machine for the therapy session. It maintains arrays for `messages`, handles Base64 conversion for `imagePreview`, manages the `currentState` for the dynamic status orb, and contains the critical `handleSend` method which builds the multimodal payload.
- **API_Gateway (represented by `apiClient.js`)**: An abstraction over the native `fetch` API. It automatically intercepts requests, attaches the Bearer JWT from `localStorage`, formats headers, and contains standardized error handling logic to prevent unhandled promise rejections.

Backend Services

- **HilaryCustomModel (represented by `GroqService` in `ai_service.py`)**:
 - **Vision Subsystem**: If `torch` is available, this class instantiates the `mental_health_image_model.pt` tensor graph onto the target device (CPU/CUDA). The `get_vision_emotion` method handles the complex ETL pipeline of decoding Base64, applying `torchvision.transforms`, and executing a forward pass (`model(input_tensor)`) without gradient tracking (`torch.no_grad()`).
 - **LLM Subsystem**: Uses the `groq` SDK to send structured prompts. It forces the output into a structured JSON schema (`response_format={"type": "json_object"}`) ensuring the `response`, `detected_sentiment`, `intensity`, and `insights` keys are strictly typed and reliable for backend parsing.
- **EmotionEngine**: The arbitration layer. The `multi_modal_fusion` method is a deterministic mathematical model. It assigns specific weights to behavioral data (0.2), text sentiment (0.3), and visual sentiment (0.25). It maps semantic strings (e.g., "Sad" = -0.6, "Happy" = 0.8) to numeric values, aggregates them, and maps the final sum to the definitive emotional string used by the rest of the application.
- **AlertService**: A decoupled observer. It is called at the end of the `chat` route execution. By abstracting this logic, the platform can easily scale to include SMS notifications, clinical dashboard alerts, or automated emergency services without bloating the core routing logic.